SIZMA TESTİ RAPORU

Rapor Tarihi	13/08/2026
Hazırlayan	Ömer Faruk Baysal
Test Türü	PortSwigger Web Academy Lab

1. SQL injection UNION attack, retrieving multiple values in a single column

Bu raporda PortSwigger Web Security Academy'de yer alan "SQL injection UNION attack, retrieving multiple values in a single column" labı ele alınmıştır.

2. Yönetici Özeti

Yapılan test sonucunda, hedef uygulamanın ürün filtreleme özelliğinde zafiyet tespit edilmiştir. Bu zafiyet, saldırganın uygulama ile ilgili çeşitli veriler ele geçirebilmesine olanak sağlayabilmektedir. Zafiyetin ivedilikle giderilmesi önerilmektedir.

3. Zafiyet Açıklaması

İlgili hedef sitenin ürün filtreleme özelliğinde zafiyet tespit edilmiştir. Bu zafiyetten yararlanıp UNION anahtar kelimesini kullanarak veritabanından veriler elde edilebilmektedir. Bu sayede veritabanındaki diğer tablolardan veri çekilebilmekte ve yetkisiz erişim ile birlikte hassas verilere erişilebilmektedir.

4. Risk Seviyesi

Risk Seviyesi	Kritik
CVSS v3.1 Skoru	9.4
CWE	CWE-89

5. Etki

Bu zafiyet ile uygulamanın veritabanı sorgusuna müdahale edilebildiği ve UNION tabanlı bir saldırı için kaç sütun olduğu, hangi sütunun metin tutabildiğini keşfedip veritabanından veri çekip, uygulamanın yetki seviyesine ve veritabanı yapılandırmasına bağlı olarak veritabanındaki tüm verilerin (kullanıcı bilgileri, parolalar vb.) ele geçirilebileceği kanıtlanmıştır. Bu zafiyet, kimlik doğrulama mekanizmasının atlatılmasına ve bazı durumlarda hedef sistemde komut çalıştırılmasına kadar ilerleyebilir.

6. Proof Of Concept

İlgili hedef siteye girildiğinde ana ekrandaki ürünler, kategori filtreleme özellikleriyle birlikte sergilenmektedir. Öncelikle veritabanının sütun sayısının bularak başlanabilir. Bunun için test etmek amacıyla herhangi bir kategori seçilebilmektedir. Test için "Gifts" kategorisi seçildiğinde ürünler bu kategoriye göre filtrelenecektir.

UNION anahtar kelimesi ile birlikte sql injection saldırısını yapabilmemiz için mevcut veritabanının sütun sayısının bilinmesi gereklidir. Bunun için deneme-yanılma yöntemini kullanacağız. Arama kutusunda “filter?category=Gifts” kısmından sonra ilk payload denemesi yapılacaktır fakat hatayla karşılaşılacaktır.

>’ UNION SELECT NULL--

Bu hata bize sütun sayısının karşılık bulamadığını göstermektedir. Yazdığımız tek NULL bir sütunu ifade etmektedir fakat bu veritabanında tek bir sütun olmadığı bu cevapla birlikte anlaşılmaktadır. Sıradaki test için 2 tane NULL yazarsak:

>’ UNION SELECT NULL,NULL—



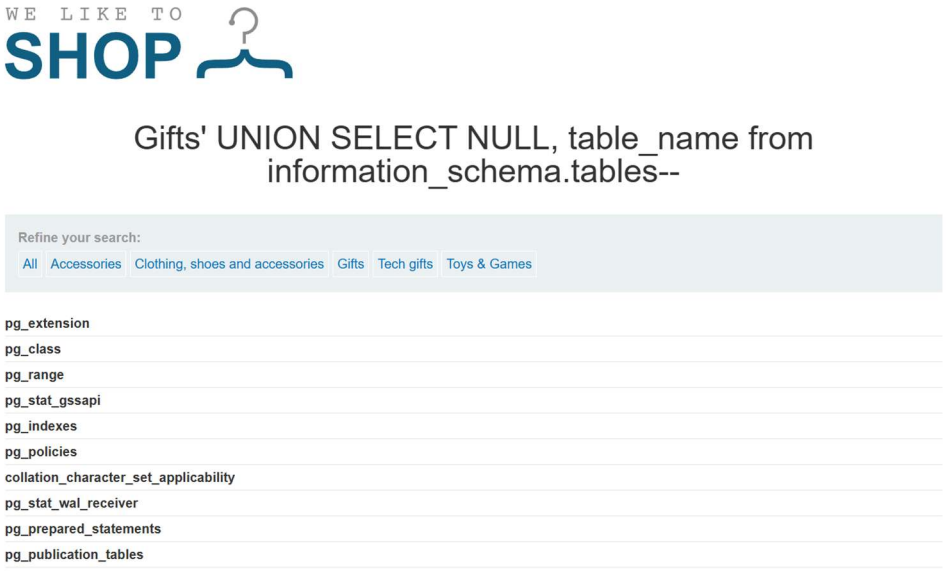
Bu denememiz bize bir çıktı-yanıt verir ve bu yüzden denememizin doğru olduğunu anlarız. Yani bu veritabanında 2 adet sütun bulunmaktadır. Böylelikle sütun sayısını bulmuş oluruz.

Hangi sütunun metin tuttuğunu bulmak için NULL kısımlarında denemeler yapılır. Bunun için NULL yerine string verilebilir.
NULL yazdığımız kısımların karşılığını test etmek amacıyla verilen rastgele stringi ilK NULL kısmında yerine yazarsak hatayla karşılaşılır. İlk kısmı NULL bırakıp ikinci kısımda deneme yaparsak çıktı başarıyla döner:



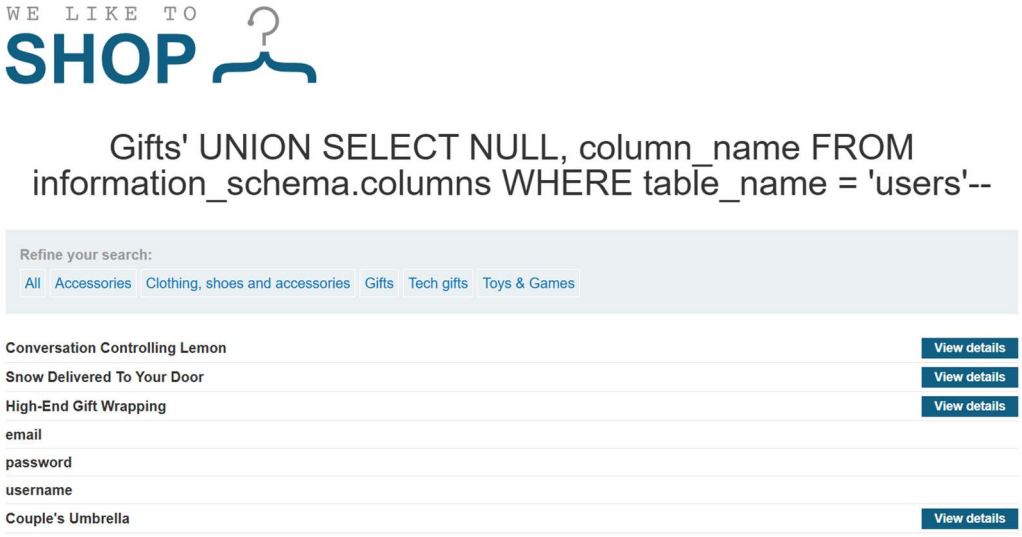
Veritabanında yer alan tüm tabloları görmek için information_schema özelliğinden yararlanılabilir.

>‘ UNION SELECT NULL, table_name from information_schema.tables--



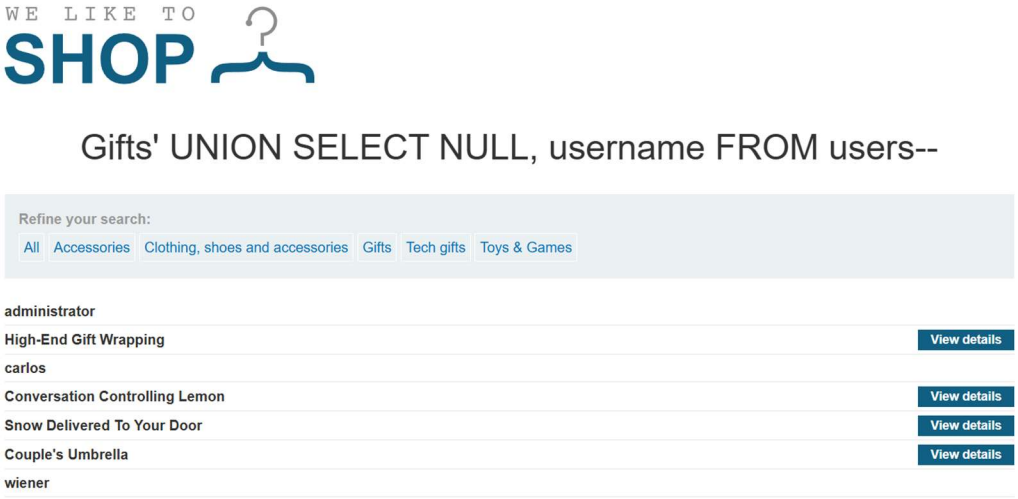
Burada bulduğumuz users tablosunu kullanıp daha derin verilere erişilebilir. Öncelikle users tablosunun yapısını daha iyi anlamak için bu tablonun sütunlarını inceleyebiliriz:

> ' UNION SELECT NULL, column_name FROM information_schema.columns WHERE table_name = 'users'—

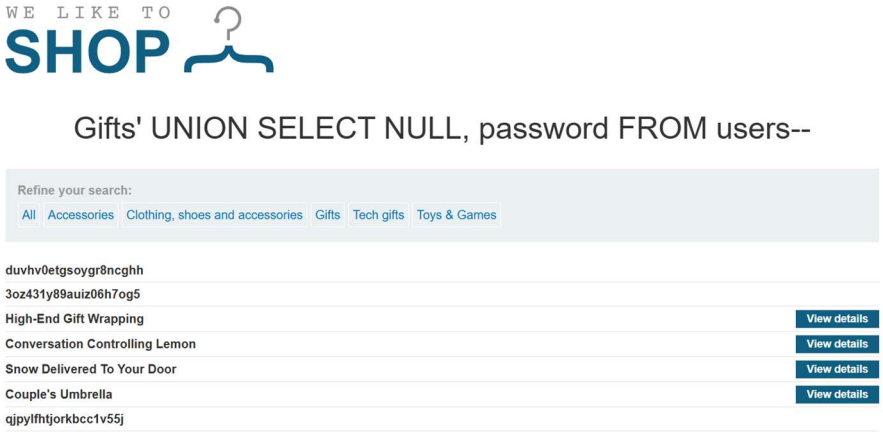


Username ve password verilerini ayrı ayrı payloadlarla döndürebiliriz:

>' UNION SELECT NULL, username FROM users--



Administrator adlı kullanıcının var olduğunu kanıtlamış oluruz.



Burada yer şifreleri administrator için deneriz. “qjpylfhtjorkbcc1v55j” şifresi ile giriş yapılabilmektedir.

Böylelikle administrator hesap bilgilerini ele geçirmiş oluruz. Verilen bilgiler ile giriş yapılabilmektedir. Bu da enjeksiyonumuzun doğrulunun kanıtı olmaktadır.

Tek bir sütun metin tutabildiği için farklı tablolardan verileri ayrı ayrı çekerek çözüme gidilmektedir.

Bu lab birleştirme (concatenation, ||) tekniğini hedeflese de, çözüm ayrı ayrı veri çekerek de sağlanabilmiştir.

Eğer || kullanarak aşağıdaki payload yazılsaydı:

>' UNION SELECT NULL, username || '~' || password FROM users—

username || '~' || password kısımları tek parça halinde çıkardı. Daha pratik ve hızlı çözüm sağlanabilirdi.

7. Çözüm Önerisi

Parametrik sorgular kullanılmalıdır. Kullanıcı girdisi, SQL sorgusuna doğrudan string olarak birleştirilmemelidir. Bunun yerine, sorgu yapısı ile kullanıcı verisini ayrı işleyen parametrik sorgular kullanılmalıdır. Bu sayede kullanıcı girdisi, SQL kodu olarak değil yalnızca veri olarak yorumlanır ve zafiyet kökten önlenir.

Uygulamanın kullandığı veritabanı hesabına yalnızca ihtiyaç duyduğu minimum yetkiler verilmelidir. Böylece olası bir injection durumunda etkinin (veri sızıntısı, komut çalıştırma) kapsamı sınırlandırılmalıdır.

8.Sonuç

Testin sonucunda hedefin sql enjeksiyonuna açık olduğu ve veritabanı hakkında bilgilere erişim sağlanabileceği tespit edilmiştir. Zafiyet kritik seviyede olup ivedilikle giderilmelidir.